

```
In [8]: import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.preprocessing import LabelEncoder
```

```
In [9]: device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print(f"Using device: {device}")
```

Using device: mps

```
In [10]: # reading the dataset & displaying the first few rows
df = pd.read_csv('hand_landmarks_data.csv')
df.head()
```

```
Out[10]:
```

|   | x1         | y1         | z1            | x2         | y2         | z2        |
|---|------------|------------|---------------|------------|------------|-----------|
| 0 | 262.669968 | 257.304901 | -3.649205e-07 | 257.417542 | 247.109055 | 0.004224  |
| 1 | 83.351778  | 346.059113 | -2.345265e-07 | 81.925037  | 328.562347 | -0.011102 |
| 2 | 187.756977 | 260.235492 | -2.417307e-07 | 195.460579 | 241.506035 | -0.000184 |
| 3 | 114.976696 | 331.594238 | -1.233261e-07 | 114.503494 | 320.549957 | -0.002824 |
| 4 | 188.795288 | 141.727867 | -1.622995e-07 | 188.520905 | 127.947464 | -0.002884 |

5 rows x 64 columns

```
In [11]: num_landmarks = 21
def normalize_landmarks(row):
    wrist_x, wrist_y = row["x1"], row["y1"]
    mid_finger_x, mid_finger_y = row["x13"], row["y13"]
    scale = np.sqrt((mid_finger_x - wrist_x) ** 2 + (mid_finger_y - wrist_y) ** 2)
    # Avoid division by zero
    if scale == 0:
        scale = 1
    for i in range(num_landmarks):
        row[f"x{i+1}"] = (row[f"x{i+1}"] - wrist_x) / scale
        row[f"y{i+1}"] = (row[f"y{i+1}"] - wrist_y) / scale
    return row
# Apply normalization
df_normalized = df.apply(normalize_landmarks, axis=1)
```

```
In [12]: label_encoder = LabelEncoder()

df_normalized["label_encoded"] = label_encoder.fit_transform(df_normalized["label"])

# Check encoding
print(df_normalized[["label", "label_encoded"]].head())

#decoded_label = label_encoder.inverse_transform([0]) # Converts 0 back

  label  label_encoded
0  call              0
1  call              0
2  call              0
3  call              0
4  call              0
```

```
In [13]: from sklearn.model_selection import train_test_split
import torch

# Separate features and labels
X = df_normalized.iloc[:, :-2].values # All columns except 'label' and 'label_encoded'
y = df_normalized["label_encoded"].values

# Split into train, validation, and test sets
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3,
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.3,

# Convert to PyTorch tensors
X_train_tensor = torch.tensor(X_train, dtype=torch.float32).to(device)
y_train_tensor = torch.tensor(y_train, dtype=torch.long).to(device)
X_val_tensor = torch.tensor(X_val, dtype=torch.float32).to(device)
y_val_tensor = torch.tensor(y_val, dtype=torch.long).to(device)
X_test_tensor = torch.tensor(X_test, dtype=torch.float32).to(device)
y_test_tensor = torch.tensor(y_test, dtype=torch.long).to(device)

print("Data prepared for PyTorch!")
```

Data prepared for PyTorch!

```
In [21]: class HaGRID(nn.Module):
    def __init__(self):
        super(HaGRID, self).__init__()
        self.fc1 = nn.Linear(63, 256) # First hidden layer
        self.fc2 = nn.Linear(256, 128) # Second hidden layer
        self.fc3 = nn.Linear(128, 64) # third hidden layer
        self.fc4 = nn.Linear(64, 18) # Output layer (18 classes)

    def forward(self, x):
        x = F.relu(self.fc1(x)) # Activation function for first hidden layer
        x = F.relu(self.fc2(x)) # Activation function for second hidden layer
        x = F.relu(self.fc3(x)) # Activation function for third hidden layer
        x = self.fc4(x) # No activation (logits for classification)
        return x
```

```
In [23]: model = HaGRID()
print(model)
```

```
HaGRID(
    (fc1): Linear(in_features=63, out_features=256, bias=True)
    (fc2): Linear(in_features=256, out_features=128, bias=True)
    (fc3): Linear(in_features=128, out_features=64, bias=True)
    (fc4): Linear(in_features=64, out_features=18, bias=True)
)
```

```
In [24]: import torch.optim as optim
criterion = nn.CrossEntropyLoss() # Loss function
optimizer = optim.Adam(model.parameters(), lr=0.0001)
```

```
In [25]: from torch.utils.data import DataLoader, TensorDataset

# Create TensorDatasets
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

# Create DataLoaders
batch_size = 64
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

```
In [28]: from sklearn.metrics import precision_score, recall_score, accuracy_score

# Move the model to the "mps" device
model = model.to(device)

# Training loop
num_epochs = 100 # You can adjust this as needed
for epoch in range(num_epochs):
    model.train() # Set model to training mode
    train_loss = 0.0
    y_true_train = []
    y_pred_train = []

    for inputs, labels in train_loader:
        # Move inputs and labels to the "mps" device
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad() # Clear gradients
        outputs = model(inputs) # Forward pass
        loss = criterion(outputs, labels) # Compute loss
        loss.backward() # Backpropagation
        optimizer.step() # Update weights

        train_loss += loss.item() * inputs.size(0) # Accumulate loss
        y_true_train.extend(labels.cpu().numpy())
        y_pred_train.extend(torch.argmax(outputs, dim=1).cpu().numpy())

    train_loss /= len(train_loader.dataset)
    train_accuracy = accuracy_score(y_true_train, y_pred_train)
    train_precision = precision_score(y_true_train, y_pred_train, average='weighted')
    train_recall = recall_score(y_true_train, y_pred_train, average='weighted')

    # Validation loop
    model.eval() # Set model to evaluation mode
    val_loss = 0.0
```

```
y_true_val = []
y_pred_val = []

with torch.no_grad():
    for inputs, labels in val_loader:
        # Move inputs and labels to the "mps" device
        inputs, labels = inputs.to(device), labels.to(device)

        outputs = model(inputs) # Forward pass
        loss = criterion(outputs, labels) # Compute loss

        val_loss += loss.item() * inputs.size(0) # Accumulate loss
        y_true_val.extend(labels.cpu().numpy())
        y_pred_val.extend(torch.argmax(outputs, dim=1).cpu().numpy())

val_loss /= len(val_loader.dataset)
val_accuracy = accuracy_score(y_true_val, y_pred_val)
val_precision = precision_score(y_true_val, y_pred_val, average='weig
val_recall = recall_score(y_true_val, y_pred_val, average='weighted',

# Print metrics
print(f"Epoch {epoch+1}/{num_epochs}")
print(f"Train Loss: {train_loss:.4f}, Accuracy: {train_accuracy:.4f},
print(f"Val Loss: {val_loss:.4f}, Accuracy: {val_accuracy:.4f}, Preci
print("-" * 50)
```

Epoch 1/100

Train Loss: 0.4161, Accuracy: 0.8640, Precision: 0.8636, Recall: 0.8640  
Val Loss: 0.3956, Accuracy: 0.8650, Precision: 0.8660, Recall: 0.8650

---

Epoch 2/100

Train Loss: 0.3829, Accuracy: 0.8726, Precision: 0.8725, Recall: 0.8726  
Val Loss: 0.3675, Accuracy: 0.8746, Precision: 0.8757, Recall: 0.8746

---

Epoch 3/100

Train Loss: 0.3589, Accuracy: 0.8779, Precision: 0.8777, Recall: 0.8779  
Val Loss: 0.3513, Accuracy: 0.8686, Precision: 0.8743, Recall: 0.8686

---

Epoch 4/100

Train Loss: 0.3404, Accuracy: 0.8837, Precision: 0.8835, Recall: 0.8837  
Val Loss: 0.3265, Accuracy: 0.8938, Precision: 0.8953, Recall: 0.8938

---

Epoch 5/100

Train Loss: 0.3249, Accuracy: 0.8887, Precision: 0.8885, Recall: 0.8887  
Val Loss: 0.3213, Accuracy: 0.8842, Precision: 0.8879, Recall: 0.8842

---

Epoch 6/100

Train Loss: 0.3137, Accuracy: 0.8913, Precision: 0.8913, Recall: 0.8913  
Val Loss: 0.3039, Accuracy: 0.9050, Precision: 0.9064, Recall: 0.9050

---

Epoch 7/100

Train Loss: 0.3024, Accuracy: 0.8949, Precision: 0.8947, Recall: 0.8949  
Val Loss: 0.2962, Accuracy: 0.8961, Precision: 0.8991, Recall: 0.8961

---

Epoch 8/100

Train Loss: 0.2907, Accuracy: 0.9037, Precision: 0.9037, Recall: 0.9037  
Val Loss: 0.2819, Accuracy: 0.9094, Precision: 0.9117, Recall: 0.9094

---

Epoch 9/100

Train Loss: 0.2808, Accuracy: 0.9105, Precision: 0.9104, Recall: 0.9105  
Val Loss: 0.2720, Accuracy: 0.9143, Precision: 0.9144, Recall: 0.9143

---

Epoch 10/100

Train Loss: 0.2724, Accuracy: 0.9159, Precision: 0.9159, Recall: 0.9159  
Val Loss: 0.2656, Accuracy: 0.9156, Precision: 0.9187, Recall: 0.9156

---

Epoch 11/100

Train Loss: 0.2623, Accuracy: 0.9203, Precision: 0.9203, Recall: 0.9203  
Val Loss: 0.2562, Accuracy: 0.9247, Precision: 0.9269, Recall: 0.9247

---

Epoch 12/100

Train Loss: 0.2520, Accuracy: 0.9277, Precision: 0.9277, Recall: 0.9277  
Val Loss: 0.2426, Accuracy: 0.9309, Precision: 0.9311, Recall: 0.9309

---

Epoch 13/100

Train Loss: 0.2439, Accuracy: 0.9297, Precision: 0.9297, Recall: 0.9297  
Val Loss: 0.2333, Accuracy: 0.9327, Precision: 0.9339, Recall: 0.9327

---

Epoch 14/100

Train Loss: 0.2326, Accuracy: 0.9375, Precision: 0.9376, Recall: 0.9375  
Val Loss: 0.2281, Accuracy: 0.9359, Precision: 0.9368, Recall: 0.9359

---

Epoch 15/100

Train Loss: 0.2220, Accuracy: 0.9419, Precision: 0.9420, Recall: 0.9419  
Val Loss: 0.2242, Accuracy: 0.9273, Precision: 0.9330, Recall: 0.9273

---

Epoch 16/100

Train Loss: 0.2121, Accuracy: 0.9468, Precision: 0.9470, Recall: 0.9468  
Val Loss: 0.2099, Accuracy: 0.9431, Precision: 0.9445, Recall: 0.9431

---

Epoch 17/100

Train Loss: 0.2019, Accuracy: 0.9500, Precision: 0.9501, Recall: 0.9500  
Val Loss: 0.2093, Accuracy: 0.9390, Precision: 0.9416, Recall: 0.9390

---

Epoch 18/100

Train Loss: 0.1927, Accuracy: 0.9541, Precision: 0.9542, Recall: 0.9541  
Val Loss: 0.1869, Accuracy: 0.9538, Precision: 0.9542, Recall: 0.9538

---

Epoch 19/100

Train Loss: 0.1847, Accuracy: 0.9560, Precision: 0.9561, Recall: 0.9560  
Val Loss: 0.1766, Accuracy: 0.9538, Precision: 0.9543, Recall: 0.9538

---

Epoch 20/100

Train Loss: 0.1749, Accuracy: 0.9599, Precision: 0.9600, Recall: 0.9599  
Val Loss: 0.1823, Accuracy: 0.9481, Precision: 0.9498, Recall: 0.9481

---

Epoch 21/100

Train Loss: 0.1681, Accuracy: 0.9617, Precision: 0.9617, Recall: 0.9617  
Val Loss: 0.1607, Accuracy: 0.9592, Precision: 0.9598, Recall: 0.9592

---

Epoch 22/100

Train Loss: 0.1616, Accuracy: 0.9639, Precision: 0.9641, Recall: 0.9639  
Val Loss: 0.1593, Accuracy: 0.9629, Precision: 0.9636, Recall: 0.9629

---

Epoch 23/100

Train Loss: 0.1549, Accuracy: 0.9652, Precision: 0.9653, Recall: 0.9652  
Val Loss: 0.1529, Accuracy: 0.9613, Precision: 0.9617, Recall: 0.9613

---

Epoch 24/100

Train Loss: 0.1492, Accuracy: 0.9676, Precision: 0.9677, Recall: 0.9676  
Val Loss: 0.1426, Accuracy: 0.9642, Precision: 0.9644, Recall: 0.9642

---

Epoch 25/100

Train Loss: 0.1436, Accuracy: 0.9681, Precision: 0.9682, Recall: 0.9681  
Val Loss: 0.1379, Accuracy: 0.9660, Precision: 0.9661, Recall: 0.9660

---

Epoch 26/100

Train Loss: 0.1389, Accuracy: 0.9697, Precision: 0.9698, Recall: 0.9697  
Val Loss: 0.1450, Accuracy: 0.9631, Precision: 0.9637, Recall: 0.9631

---

Epoch 27/100

Train Loss: 0.1352, Accuracy: 0.9703, Precision: 0.9704, Recall: 0.9703  
Val Loss: 0.1300, Accuracy: 0.9691, Precision: 0.9695, Recall: 0.9691

---

Epoch 28/100

Train Loss: 0.1301, Accuracy: 0.9712, Precision: 0.9713, Recall: 0.9712  
Val Loss: 0.1260, Accuracy: 0.9694, Precision: 0.9697, Recall: 0.9694

---

Epoch 29/100

Train Loss: 0.1266, Accuracy: 0.9734, Precision: 0.9735, Recall: 0.9734  
Val Loss: 0.1234, Accuracy: 0.9714, Precision: 0.9717, Recall: 0.9714

---

Epoch 30/100

Train Loss: 0.1234, Accuracy: 0.9728, Precision: 0.9729, Recall: 0.9728  
Val Loss: 0.1192, Accuracy: 0.9707, Precision: 0.9710, Recall: 0.9707

---

```
Epoch 31/100
Train Loss: 0.1200, Accuracy: 0.9739, Precision: 0.9739, Recall: 0.9739
Val Loss: 0.1180, Accuracy: 0.9712, Precision: 0.9717, Recall: 0.9712
-----
Epoch 32/100
Train Loss: 0.1163, Accuracy: 0.9747, Precision: 0.9748, Recall: 0.9747
Val Loss: 0.1167, Accuracy: 0.9691, Precision: 0.9694, Recall: 0.9691
-----
Epoch 33/100
Train Loss: 0.1140, Accuracy: 0.9752, Precision: 0.9753, Recall: 0.9752
Val Loss: 0.1116, Accuracy: 0.9725, Precision: 0.9728, Recall: 0.9725
-----
Epoch 34/100
Train Loss: 0.1108, Accuracy: 0.9763, Precision: 0.9764, Recall: 0.9763
Val Loss: 0.1077, Accuracy: 0.9740, Precision: 0.9742, Recall: 0.9740
-----
Epoch 35/100
Train Loss: 0.1079, Accuracy: 0.9759, Precision: 0.9759, Recall: 0.9759
Val Loss: 0.1098, Accuracy: 0.9748, Precision: 0.9752, Recall: 0.9748
-----
Epoch 36/100
Train Loss: 0.1061, Accuracy: 0.9773, Precision: 0.9774, Recall: 0.9773
Val Loss: 0.1036, Accuracy: 0.9753, Precision: 0.9757, Recall: 0.9753
-----
Epoch 37/100
Train Loss: 0.1035, Accuracy: 0.9784, Precision: 0.9785, Recall: 0.9784
Val Loss: 0.1019, Accuracy: 0.9771, Precision: 0.9774, Recall: 0.9771
-----
Epoch 38/100
Train Loss: 0.1004, Accuracy: 0.9783, Precision: 0.9783, Recall: 0.9783
Val Loss: 0.0988, Accuracy: 0.9774, Precision: 0.9776, Recall: 0.9774
-----
Epoch 39/100
Train Loss: 0.0988, Accuracy: 0.9790, Precision: 0.9791, Recall: 0.9790
Val Loss: 0.1028, Accuracy: 0.9787, Precision: 0.9790, Recall: 0.9787
-----
Epoch 40/100
Train Loss: 0.0964, Accuracy: 0.9790, Precision: 0.9791, Recall: 0.9790
Val Loss: 0.0991, Accuracy: 0.9766, Precision: 0.9768, Recall: 0.9766
-----
Epoch 41/100
Train Loss: 0.0951, Accuracy: 0.9795, Precision: 0.9796, Recall: 0.9795
Val Loss: 0.0915, Accuracy: 0.9797, Precision: 0.9799, Recall: 0.9797
-----
Epoch 42/100
Train Loss: 0.0923, Accuracy: 0.9795, Precision: 0.9796, Recall: 0.9795
Val Loss: 0.0933, Accuracy: 0.9803, Precision: 0.9805, Recall: 0.9803
-----
Epoch 43/100
Train Loss: 0.0913, Accuracy: 0.9794, Precision: 0.9795, Recall: 0.9794
Val Loss: 0.0905, Accuracy: 0.9790, Precision: 0.9791, Recall: 0.9790
-----
Epoch 44/100
Train Loss: 0.0892, Accuracy: 0.9805, Precision: 0.9806, Recall: 0.9805
Val Loss: 0.0906, Accuracy: 0.9795, Precision: 0.9799, Recall: 0.9795
-----
Epoch 45/100
Train Loss: 0.0876, Accuracy: 0.9801, Precision: 0.9802, Recall: 0.9801
Val Loss: 0.0942, Accuracy: 0.9759, Precision: 0.9766, Recall: 0.9759
-----
```

```
Epoch 46/100
Train Loss: 0.0856, Accuracy: 0.9810, Precision: 0.9811, Recall: 0.9810
Val Loss: 0.0865, Accuracy: 0.9816, Precision: 0.9817, Recall: 0.9816
-----
Epoch 47/100
Train Loss: 0.0845, Accuracy: 0.9814, Precision: 0.9815, Recall: 0.9814
Val Loss: 0.0844, Accuracy: 0.9821, Precision: 0.9822, Recall: 0.9821
-----
Epoch 48/100
Train Loss: 0.0832, Accuracy: 0.9811, Precision: 0.9811, Recall: 0.9811
Val Loss: 0.0852, Accuracy: 0.9821, Precision: 0.9823, Recall: 0.9821
-----
Epoch 49/100
Train Loss: 0.0818, Accuracy: 0.9819, Precision: 0.9819, Recall: 0.9819
Val Loss: 0.0841, Accuracy: 0.9831, Precision: 0.9833, Recall: 0.9831
-----
Epoch 50/100
Train Loss: 0.0797, Accuracy: 0.9819, Precision: 0.9819, Recall: 0.9819
Val Loss: 0.0831, Accuracy: 0.9816, Precision: 0.9818, Recall: 0.9816
-----
Epoch 51/100
Train Loss: 0.0785, Accuracy: 0.9823, Precision: 0.9823, Recall: 0.9823
Val Loss: 0.0837, Accuracy: 0.9821, Precision: 0.9824, Recall: 0.9821
-----
Epoch 52/100
Train Loss: 0.0778, Accuracy: 0.9816, Precision: 0.9816, Recall: 0.9816
Val Loss: 0.0829, Accuracy: 0.9821, Precision: 0.9825, Recall: 0.9821
-----
Epoch 53/100
Train Loss: 0.0759, Accuracy: 0.9824, Precision: 0.9825, Recall: 0.9824
Val Loss: 0.0813, Accuracy: 0.9829, Precision: 0.9831, Recall: 0.9829
-----
Epoch 54/100
Train Loss: 0.0752, Accuracy: 0.9825, Precision: 0.9826, Recall: 0.9825
Val Loss: 0.0789, Accuracy: 0.9831, Precision: 0.9833, Recall: 0.9831
-----
Epoch 55/100
Train Loss: 0.0738, Accuracy: 0.9826, Precision: 0.9826, Recall: 0.9826
Val Loss: 0.0782, Accuracy: 0.9847, Precision: 0.9849, Recall: 0.9847
-----
Epoch 56/100
Train Loss: 0.0731, Accuracy: 0.9830, Precision: 0.9830, Recall: 0.9830
Val Loss: 0.0769, Accuracy: 0.9847, Precision: 0.9848, Recall: 0.9847
-----
Epoch 57/100
Train Loss: 0.0717, Accuracy: 0.9839, Precision: 0.9840, Recall: 0.9839
Val Loss: 0.0778, Accuracy: 0.9839, Precision: 0.9840, Recall: 0.9839
-----
Epoch 58/100
Train Loss: 0.0708, Accuracy: 0.9836, Precision: 0.9837, Recall: 0.9836
Val Loss: 0.0756, Accuracy: 0.9839, Precision: 0.9841, Recall: 0.9839
-----
Epoch 59/100
Train Loss: 0.0702, Accuracy: 0.9830, Precision: 0.9831, Recall: 0.9830
Val Loss: 0.0789, Accuracy: 0.9826, Precision: 0.9828, Recall: 0.9826
-----
Epoch 60/100
Train Loss: 0.0688, Accuracy: 0.9835, Precision: 0.9835, Recall: 0.9835
Val Loss: 0.0759, Accuracy: 0.9831, Precision: 0.9833, Recall: 0.9831
-----
```

Epoch 61/100

Train Loss: 0.0687, Accuracy: 0.9839, Precision: 0.9839, Recall: 0.9839  
Val Loss: 0.0731, Accuracy: 0.9844, Precision: 0.9846, Recall: 0.9844

---

Epoch 62/100

Train Loss: 0.0673, Accuracy: 0.9845, Precision: 0.9846, Recall: 0.9845  
Val Loss: 0.0756, Accuracy: 0.9844, Precision: 0.9846, Recall: 0.9844

---

Epoch 63/100

Train Loss: 0.0667, Accuracy: 0.9846, Precision: 0.9847, Recall: 0.9846  
Val Loss: 0.0721, Accuracy: 0.9852, Precision: 0.9853, Recall: 0.9852

---

Epoch 64/100

Train Loss: 0.0658, Accuracy: 0.9847, Precision: 0.9847, Recall: 0.9847  
Val Loss: 0.0723, Accuracy: 0.9847, Precision: 0.9848, Recall: 0.9847

---

Epoch 65/100

Train Loss: 0.0645, Accuracy: 0.9850, Precision: 0.9850, Recall: 0.9850  
Val Loss: 0.0739, Accuracy: 0.9829, Precision: 0.9830, Recall: 0.9829

---

Epoch 66/100

Train Loss: 0.0646, Accuracy: 0.9845, Precision: 0.9846, Recall: 0.9845  
Val Loss: 0.0718, Accuracy: 0.9862, Precision: 0.9864, Recall: 0.9862

---

Epoch 67/100

Train Loss: 0.0630, Accuracy: 0.9854, Precision: 0.9855, Recall: 0.9854  
Val Loss: 0.0719, Accuracy: 0.9844, Precision: 0.9846, Recall: 0.9844

---

Epoch 68/100

Train Loss: 0.0626, Accuracy: 0.9855, Precision: 0.9855, Recall: 0.9855  
Val Loss: 0.0705, Accuracy: 0.9847, Precision: 0.9849, Recall: 0.9847

---

Epoch 69/100

Train Loss: 0.0626, Accuracy: 0.9847, Precision: 0.9847, Recall: 0.9847  
Val Loss: 0.0719, Accuracy: 0.9847, Precision: 0.9849, Recall: 0.9847

---

Epoch 70/100

Train Loss: 0.0610, Accuracy: 0.9857, Precision: 0.9857, Recall: 0.9857  
Val Loss: 0.0715, Accuracy: 0.9849, Precision: 0.9851, Recall: 0.9849

---

Epoch 71/100

Train Loss: 0.0607, Accuracy: 0.9859, Precision: 0.9860, Recall: 0.9859  
Val Loss: 0.0690, Accuracy: 0.9842, Precision: 0.9843, Recall: 0.9842

---

Epoch 72/100

Train Loss: 0.0605, Accuracy: 0.9855, Precision: 0.9856, Recall: 0.9855  
Val Loss: 0.0711, Accuracy: 0.9834, Precision: 0.9836, Recall: 0.9834

---

Epoch 73/100

Train Loss: 0.0604, Accuracy: 0.9860, Precision: 0.9860, Recall: 0.9860  
Val Loss: 0.0696, Accuracy: 0.9857, Precision: 0.9859, Recall: 0.9857

---

Epoch 74/100

Train Loss: 0.0589, Accuracy: 0.9860, Precision: 0.9861, Recall: 0.9860  
Val Loss: 0.0682, Accuracy: 0.9842, Precision: 0.9843, Recall: 0.9842

---

Epoch 75/100

Train Loss: 0.0584, Accuracy: 0.9857, Precision: 0.9857, Recall: 0.9857  
Val Loss: 0.0725, Accuracy: 0.9847, Precision: 0.9849, Recall: 0.9847

---

```
Epoch 76/100
Train Loss: 0.0571, Accuracy: 0.9869, Precision: 0.9870, Recall: 0.9869
Val Loss: 0.0678, Accuracy: 0.9849, Precision: 0.9851, Recall: 0.9849
-----
Epoch 77/100
Train Loss: 0.0569, Accuracy: 0.9858, Precision: 0.9858, Recall: 0.9858
Val Loss: 0.0697, Accuracy: 0.9842, Precision: 0.9843, Recall: 0.9842
-----
Epoch 78/100
Train Loss: 0.0568, Accuracy: 0.9861, Precision: 0.9862, Recall: 0.9861
Val Loss: 0.0687, Accuracy: 0.9862, Precision: 0.9863, Recall: 0.9862
-----
Epoch 79/100
Train Loss: 0.0563, Accuracy: 0.9863, Precision: 0.9863, Recall: 0.9863
Val Loss: 0.0674, Accuracy: 0.9844, Precision: 0.9846, Recall: 0.9844
-----
Epoch 80/100
Train Loss: 0.0563, Accuracy: 0.9859, Precision: 0.9860, Recall: 0.9859
Val Loss: 0.0679, Accuracy: 0.9842, Precision: 0.9843, Recall: 0.9842
-----
Epoch 81/100
Train Loss: 0.0547, Accuracy: 0.9870, Precision: 0.9871, Recall: 0.9870
Val Loss: 0.0687, Accuracy: 0.9836, Precision: 0.9839, Recall: 0.9836
-----
Epoch 82/100
Train Loss: 0.0541, Accuracy: 0.9872, Precision: 0.9872, Recall: 0.9872
Val Loss: 0.0676, Accuracy: 0.9849, Precision: 0.9851, Recall: 0.9849
-----
Epoch 83/100
Train Loss: 0.0545, Accuracy: 0.9878, Precision: 0.9878, Recall: 0.9878
Val Loss: 0.0659, Accuracy: 0.9865, Precision: 0.9866, Recall: 0.9865
-----
Epoch 84/100
Train Loss: 0.0539, Accuracy: 0.9869, Precision: 0.9870, Recall: 0.9869
Val Loss: 0.0718, Accuracy: 0.9834, Precision: 0.9836, Recall: 0.9834
-----
Epoch 85/100
Train Loss: 0.0533, Accuracy: 0.9866, Precision: 0.9866, Recall: 0.9866
Val Loss: 0.0655, Accuracy: 0.9862, Precision: 0.9864, Recall: 0.9862
-----
Epoch 86/100
Train Loss: 0.0530, Accuracy: 0.9868, Precision: 0.9868, Recall: 0.9868
Val Loss: 0.0669, Accuracy: 0.9862, Precision: 0.9864, Recall: 0.9862
-----
Epoch 87/100
Train Loss: 0.0522, Accuracy: 0.9876, Precision: 0.9877, Recall: 0.9876
Val Loss: 0.0684, Accuracy: 0.9862, Precision: 0.9864, Recall: 0.9862
-----
Epoch 88/100
Train Loss: 0.0528, Accuracy: 0.9868, Precision: 0.9868, Recall: 0.9868
Val Loss: 0.0655, Accuracy: 0.9865, Precision: 0.9866, Recall: 0.9865
-----
Epoch 89/100
Train Loss: 0.0516, Accuracy: 0.9866, Precision: 0.9867, Recall: 0.9866
Val Loss: 0.0647, Accuracy: 0.9860, Precision: 0.9861, Recall: 0.9860
-----
Epoch 90/100
Train Loss: 0.0519, Accuracy: 0.9876, Precision: 0.9876, Recall: 0.9876
Val Loss: 0.0695, Accuracy: 0.9839, Precision: 0.9842, Recall: 0.9839
-----
```

```

Epoch 91/100
Train Loss: 0.0515, Accuracy: 0.9875, Precision: 0.9875, Recall: 0.9875
Val Loss: 0.0645, Accuracy: 0.9865, Precision: 0.9866, Recall: 0.9865
-----
Epoch 92/100
Train Loss: 0.0508, Accuracy: 0.9878, Precision: 0.9878, Recall: 0.9878
Val Loss: 0.0647, Accuracy: 0.9852, Precision: 0.9854, Recall: 0.9852
-----
Epoch 93/100
Train Loss: 0.0503, Accuracy: 0.9877, Precision: 0.9877, Recall: 0.9877
Val Loss: 0.0640, Accuracy: 0.9868, Precision: 0.9869, Recall: 0.9868
-----
Epoch 94/100
Train Loss: 0.0497, Accuracy: 0.9878, Precision: 0.9878, Recall: 0.9878
Val Loss: 0.0676, Accuracy: 0.9855, Precision: 0.9856, Recall: 0.9855
-----
Epoch 95/100
Train Loss: 0.0491, Accuracy: 0.9878, Precision: 0.9878, Recall: 0.9878
Val Loss: 0.0627, Accuracy: 0.9865, Precision: 0.9866, Recall: 0.9865
-----
Epoch 96/100
Train Loss: 0.0489, Accuracy: 0.9876, Precision: 0.9877, Recall: 0.9876
Val Loss: 0.0634, Accuracy: 0.9852, Precision: 0.9854, Recall: 0.9852
-----
Epoch 97/100
Train Loss: 0.0491, Accuracy: 0.9878, Precision: 0.9878, Recall: 0.9878
Val Loss: 0.0651, Accuracy: 0.9847, Precision: 0.9849, Recall: 0.9847
-----
Epoch 98/100
Train Loss: 0.0485, Accuracy: 0.9882, Precision: 0.9882, Recall: 0.9882
Val Loss: 0.0636, Accuracy: 0.9870, Precision: 0.9871, Recall: 0.9870
-----
Epoch 99/100
Train Loss: 0.0474, Accuracy: 0.9880, Precision: 0.9880, Recall: 0.9880
Val Loss: 0.0693, Accuracy: 0.9847, Precision: 0.9850, Recall: 0.9847
-----
Epoch 100/100
Train Loss: 0.0486, Accuracy: 0.9877, Precision: 0.9877, Recall: 0.9877
Val Loss: 0.0649, Accuracy: 0.9860, Precision: 0.9862, Recall: 0.9860
-----

```

```

In [31]: # Ensure the model is in evaluation mode
         model.eval()

         # Initialize lists to store true labels and predictions
         y_true_test = []
         y_pred_test = []

         # Iterate through the test data loader
         with torch.no_grad():
             for inputs, labels in test_loader:
                 # Move inputs and labels to the device
                 inputs, labels = inputs.to(device), labels.to(device)

                 # Make predictions
                 outputs = model(inputs)
                 predictions = torch.argmax(outputs, dim=1)

                 # Append true labels and predictions to the respective lists

```

```
y_true_test.extend(labels.cpu().numpy())
y_pred_test.extend(predictions.cpu().numpy())
```

```
In [32]: from sklearn.metrics import accuracy_score, precision_score, recall_score

# Calculate test metrics
test_accuracy = accuracy_score(y_true_test, y_pred_test)
test_precision = precision_score(y_true_test, y_pred_test, average='weigh
test_recall = recall_score(y_true_test, y_pred_test, average='weighted',

# Print test metrics
print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Test Precision: {test_precision:.4f}")
print(f"Test Recall: {test_recall:.4f}")
```

Test Accuracy: 0.9829

Test Precision: 0.9830

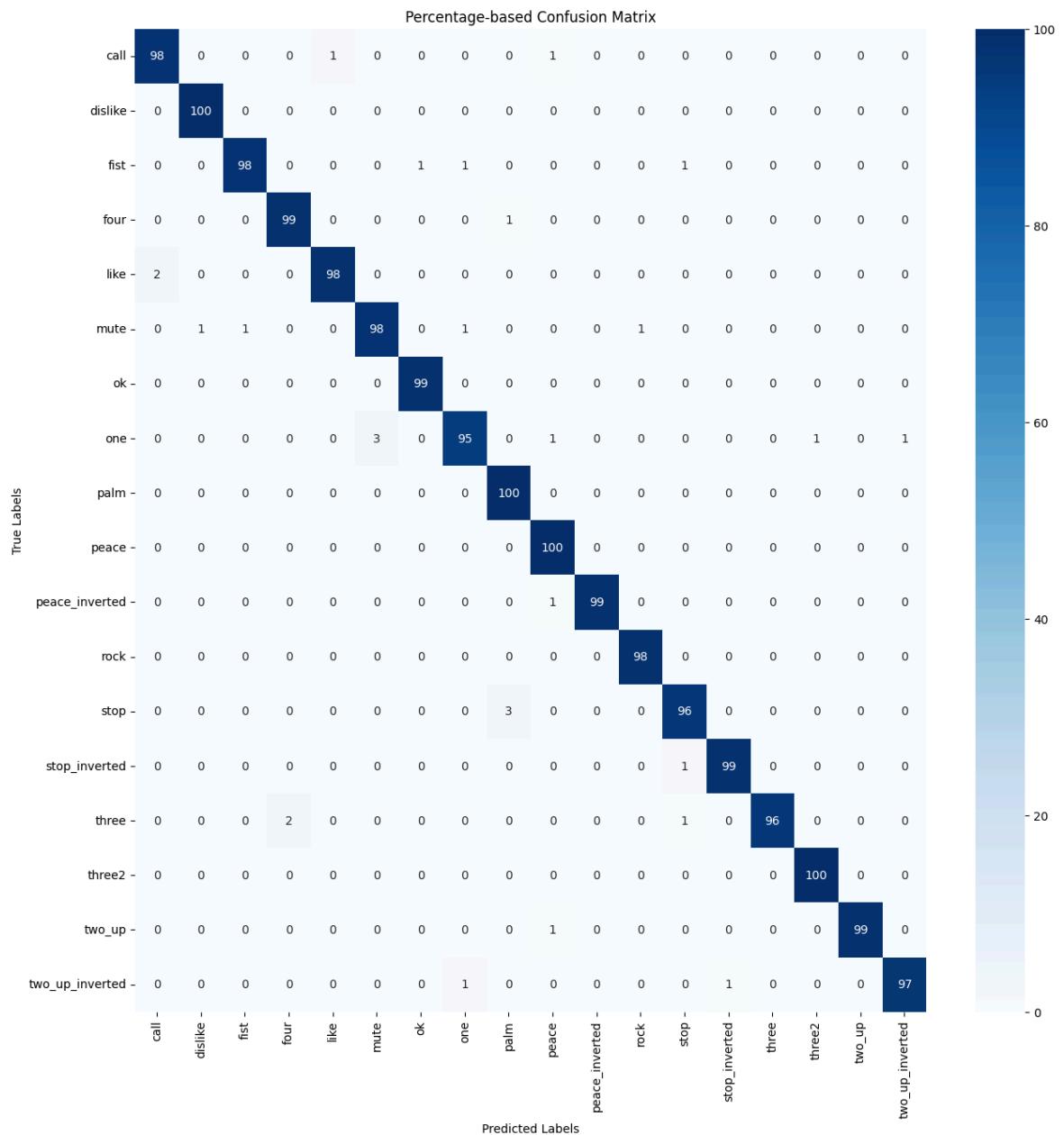
Test Recall: 0.9829

```
In [36]: from sklearn.metrics import confusion_matrix
import seaborn as sns

# Compute the confusion matrix
cm = confusion_matrix(y_true_test, y_pred_test)

# Normalize the confusion matrix to percentages
cm_percentage = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis] * 100

# Plot the confusion matrix
plt.figure(figsize=(15, 15))
sns.heatmap(cm_percentage, annot=True, fmt=".0f", cmap="Blues", xticklabel
plt.title("Percentage-based Confusion Matrix")
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")
plt.show()
```



## Comparison of Results: Neural Network vs Random Forest

### Neural Network Results

- **Test Accuracy:** 0.9829
- **Test Recall:** 0.9829
- **Test Precision:** 0.9830

### Random Forest Results

- **Test Accuracy:** 0.9715
- **Test Recall:** 0.9770
- **Test Precision:** 0.9771

## Observations

1. **Accuracy:** The neural network outperforms the random forest model with a higher accuracy (0.9829 vs 0.9715).
2. **Recall:** The neural network achieves a slightly better recall compared to the random forest (0.9829 vs 0.9770).
3. **Precision:** The neural network also has a marginally better precision than the random forest (0.9830 vs 0.9771).

Overall, the neural network demonstrates superior performance across all metrics compared to the random forest model.

below is the confusion matrix for random forest model

